Lab 3 Cross-Site Scripting (XSS) Attack

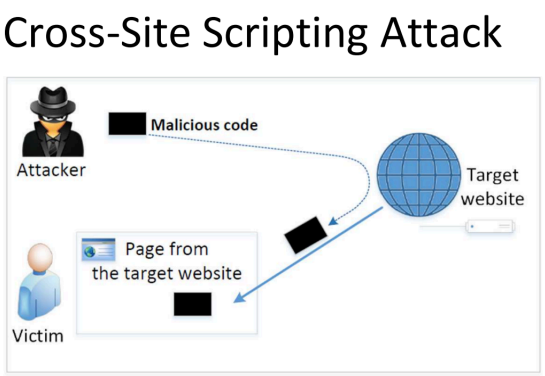
Student： *YuChin Chiang u5693885*

Date： 15/11/2025

Environment： school remote Vm

Background

1) Cross-site scripting attack work with a diagram



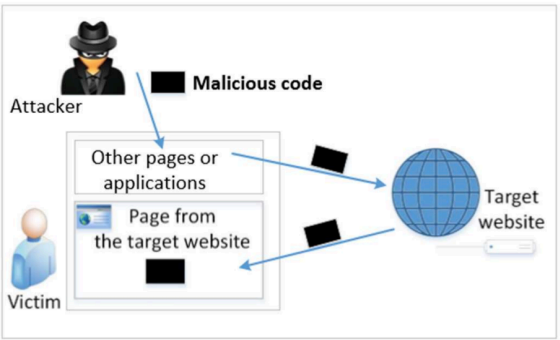
- XSS is actually “same-site” attack
- CSRF works “cross-site”

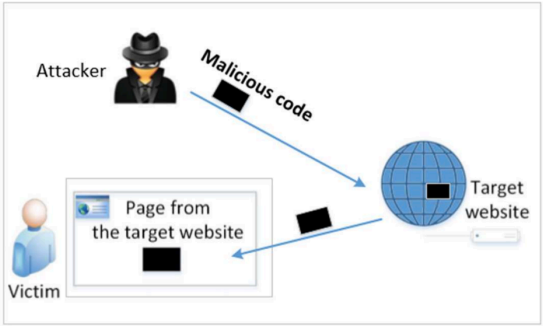
In XSS, an attacker injects malicious code into the victim's browser while they are on a browseable site. If code comes from a site, that site is trusted with respect to that code. Therefore, it has access to the page's content and is able to read cookies for the same domain, and so on.

Types of XSS Attacks

1. Non-persistent (Reflected) XSS Attack

   A website is said to be reflective if it echoes users' inputs back to them. The threat is that attackers can place JavaScript in the input, which when reflected back will cause the JavaScript to be injected into a website from its own site base.





2. Persistent (Stored) XSS Attack

Attackers make their data run through the targeted website or server, which then writes this data into some form of permanent storage. It creates a bridge to the attackers between other users when the data is later shared on another website.

---

2) Countermeasures Against XSS

2.1 Output Encoding

Convert special characters into safe HTML entities:

```
<script> → &lt;script&gt;
"        → &quot;
<        → &lt;
>        → &gt;
```

Why?

If one encodes the browser displays it as text, not executing it as JavaScript. The result is that the attacker's code doesn't run. And what we did in Task 5 is nothing else but the latter presented principle:

```
htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
```

2.2 Input Validation / Sanitization

Reject or clean dangerous characters (`< > " ' /`) before storing the input.

Why? If scripts cannot be stored in the database, they cannot be delivered to other users.

2.3 Content Security Policy (CSP)

A server header that restricts where JavaScript is allowed to load from.

Examples:

```
script-src 'self'
script-src 'self' 'nonce-xyz123'
script-src 'self' example60.com
```

Why?

Blocks inline tags , prevents scripts from untrusted domains and must be a valid nonce or hash value that this countermeasures work.

But even if an attacker injects a tag, it will not be executed on the browser because of the Content Security Policy (CSP) which I have mentioned above.

2.4 Disable Inline JavaScript

Disallow:

- `<script>`
- `onclick=...`
- `<img onerror=...>`
- Inline event handlers

Why? Most XSS worms and payloads rely on inline scripts.

2.5 HttpOnly Cookies

Mark session cookies as `HttpOnly`.

Why? JavaScript cannot access `document.cookie`, preventing session theft.

Task 1

1) Steps

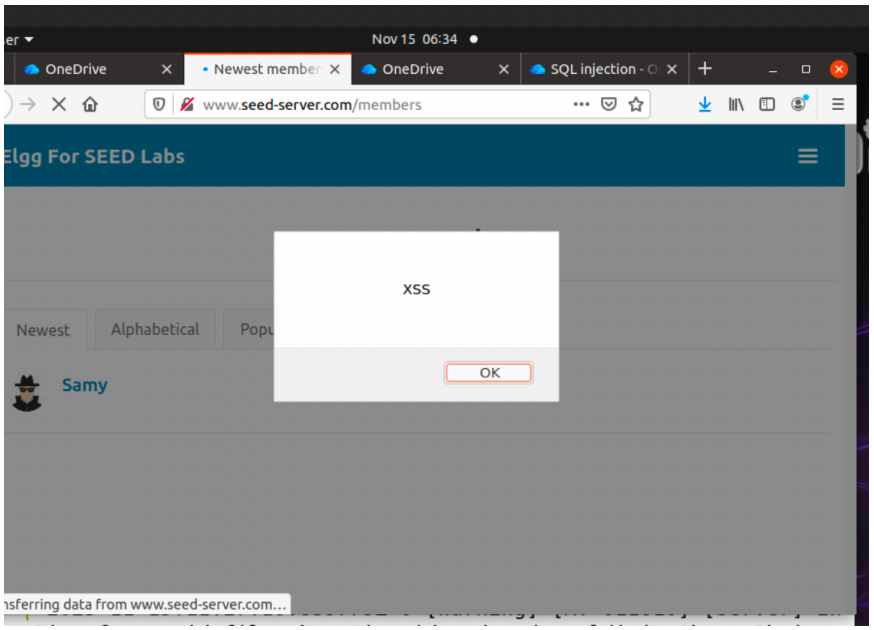
1. Log in as Samy
2. Insert the following into “Brief description”:

   ```
   <script>alert('XSS');</script>
   ```

3. Save the profile

4. Log in as Alice and visit Samy's profile

2) Results



3) Why the Attack Succeeded

Elgg allows to save input with HTML directly in its database and it doesn't escape this data.

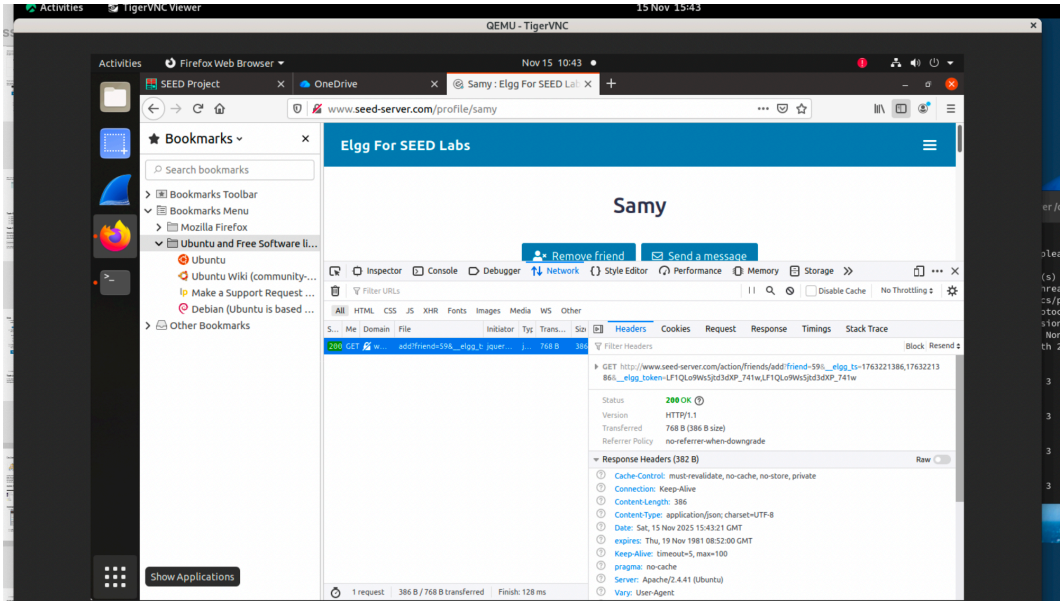
When Alice views Samy's profile, the HTML returns to her browser exactly as it is filed away. I mean is interpreted as JS, so the alert box appears.

Because the malicious script runs in Alice's login session, it has full access to all of Elgg's authenticated APIs.

Task 2

1) Observing the Friend-Add Request

Because new vision Firebox couldn't allow HTTP Header Live, so I used web developer to find informations.



Using Developer Tools , observe this request:

```
/action/friends/add?friend=59&__elgg_ts=XXXX&__elgg_token=YYYY
```

Where:

- `friend=59` → Samy's GUID
- `__elgg_ts` → timestamp
- `__elgg_token` → CSRF token

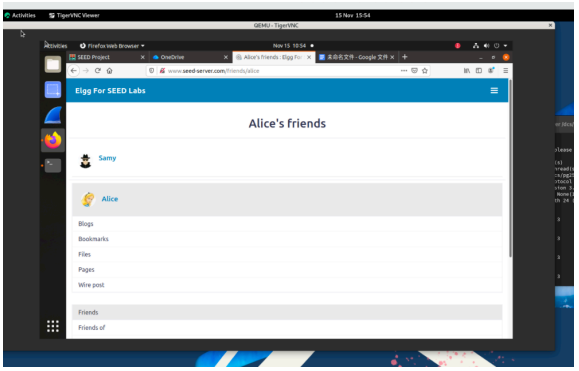
---

2) Injecting the Attack Script into Samy's Profile

Embed JavaScript that automatically sends the same GET request when Alice views Samy's profile.

3) Results

After Alice views Samy's page, Samy appears in her friend list.



4) Why the Attack Works

- The malicious script is running **in the context of Alice's authenticated session.
- Alice's CSRF token and timestamp are retrieved using:

```
elgg.security.token.__elgg_token
elgg.security.token.__elgg_ts
```

- The browser automatically adds Alice's cookies, resulting in a valid request.

This will bypass CSRF security because the script gets executed from the trusted domain!

5) Explain the purpose of (1) and (2), why are they needed? (Question 1)

(1)

```
var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
```

(2)

```
var token="&__elgg_token="+elgg.security.token.__elgg_token;
```

Answer

They are also Elgg's anti-CSRF defence fields: Both __elgg_ts and __elgg_token.

Because Elgg denies any request that does not provide timestamp and token which pairs with a user's session they are important.

Purpose:

Field	Meaning	Why Needed
`__elgg_ts`	Timestamp	Prevents replay attacks. Ensures the request is recent.

Field	Meaning	Why Needed
`__elgg_token`	CSRF token	Ensures the request was generated by the logged-in user.

Why the attacker must include them?

Because the malicious script runs inside the victim's browser, it can steal:

```
elgg.security.token.__elgg_ts
elgg.security.token.__elgg_token
```

Now craft a technically correct request that server will honour.

Without these two parameters:

- The request is blocked by the server.
- The friend-adding attack fails.
- The POST we see in Task 3Doesn't work as well.

---

5) If rich-text editor (no Edit HTML mode) is the ONLY option, can you still launch an attack? (Question 2)

Answer

It depends, but generally the attack becomes much harder or even impossible.

The Rich Text Editor **escapes dangerous characters**:

```
<script> becomes &lt;script&gt;
onclick="..." becomes text
```

Meaning the browser will **display** `<script>` as text, instead of executing it.

Therefore:

Scenario	Can attack?	Reason
Elgg escapes all tags properly	No	Script cannot execute. Stored XSS prevented.

Scenario	Can attack?	Reason
Elgg fails to sanitize some attributes	Maybe?	Could inject payload using attributes like `<img src=x onerror=...>` if not filtered.
Classic Elgg sanitization (SEED Lab patched)	Yes	Because SEED Lab intentionally removed filters.

6) Why the Rich Text Editor Cannot Perform XSS

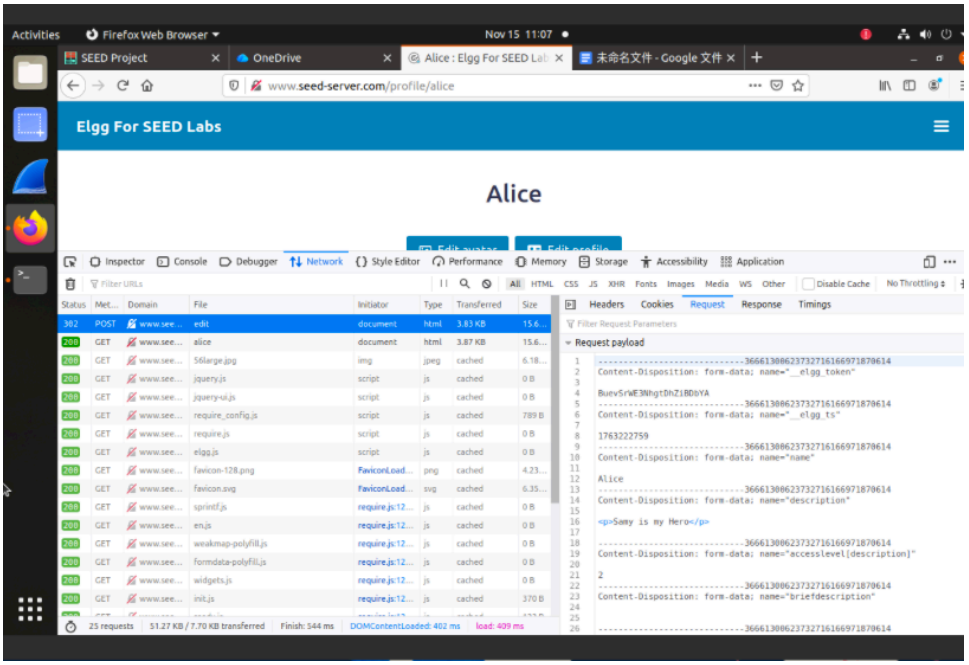
Without "Edit HTML" the special characters are encoded by the text editor. is getting escaped, so it's shown as text and does not run. Consequently, the attack is unsuccessful.

Task 3

This attack automatically changes Alice's profile information when she views Samy's infected profile.

1) Steps

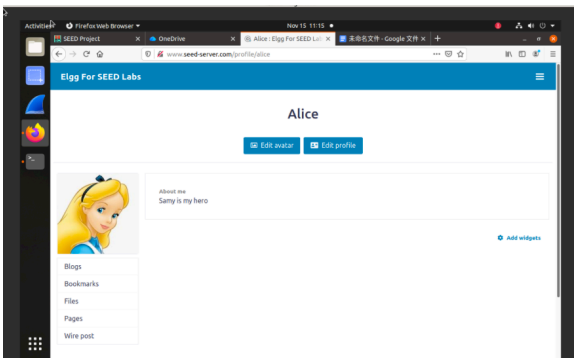
1. Capture the POST request when editing Alice's profile
2. Extract the required fields:
    - `__elgg_token`
    - `__elgg_ts`
    - `name`
    - `description`
    - `guid`
3. Build a JavaScript script that sends the same POST request
4. Inject script into Samy's profile



2) Results

When Alice views Samy's profile, her "About me" becomes:

"Samy is my hero"



3) Why We Need the Condition `if(guid != samyGuid)` (Question 3)

Without this line:

```
if (elgg.session.user.guid != 59)
```

In addition, Samy's script would not only change the Samy's own profile while on his page which becomes

- Recursion
- Profile overwrite
- May get some that are fatal

This line ensures that only OTHER users are infected.

Task 4

This task implements a worm that:

1. Modifies the victim's profile
2. Replicates itself into the victim's profile
3. Turns the victim into a new attacker
4. Spreads to every user who views an infected profile

---

1) How the Worm Replicates

Using DOM access:

```
var jsCode = document.getElementById("worm").innerHTML;
```

This extracts the worm’s body.

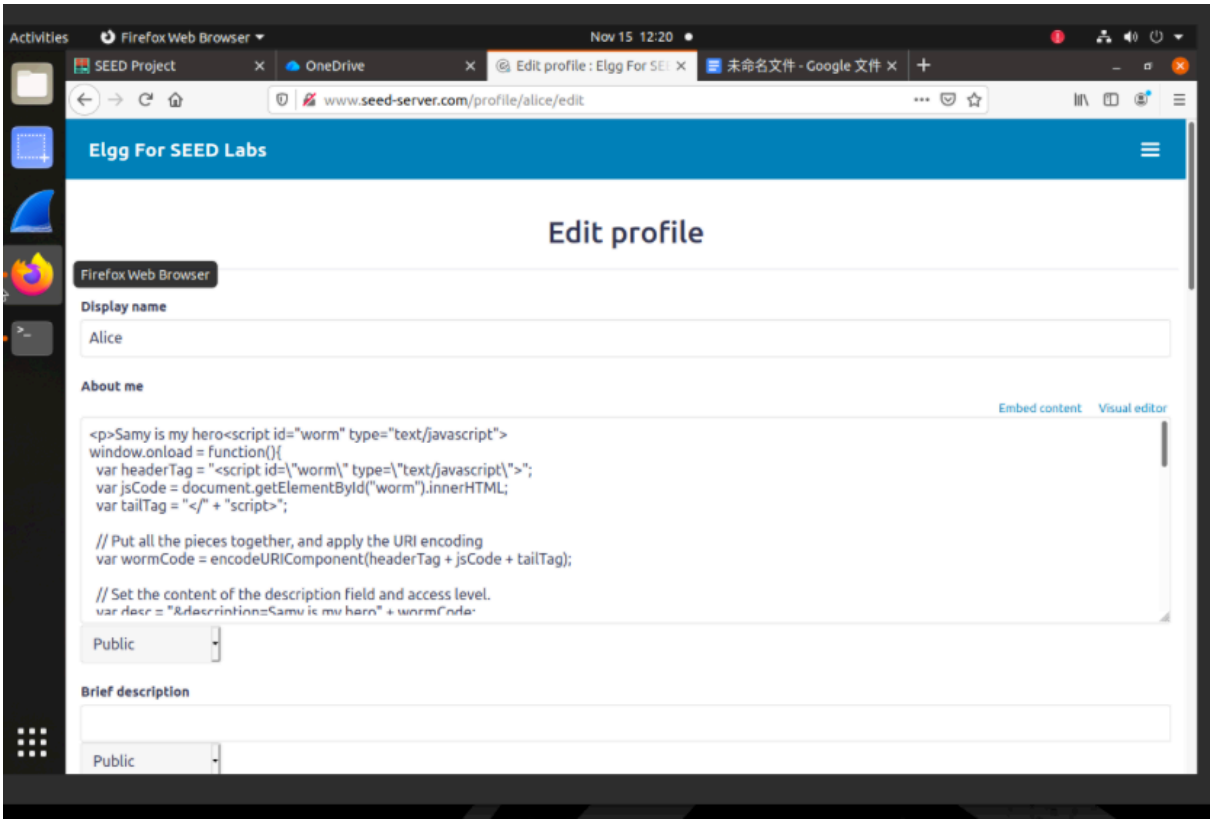
Then reconstructs the full script:

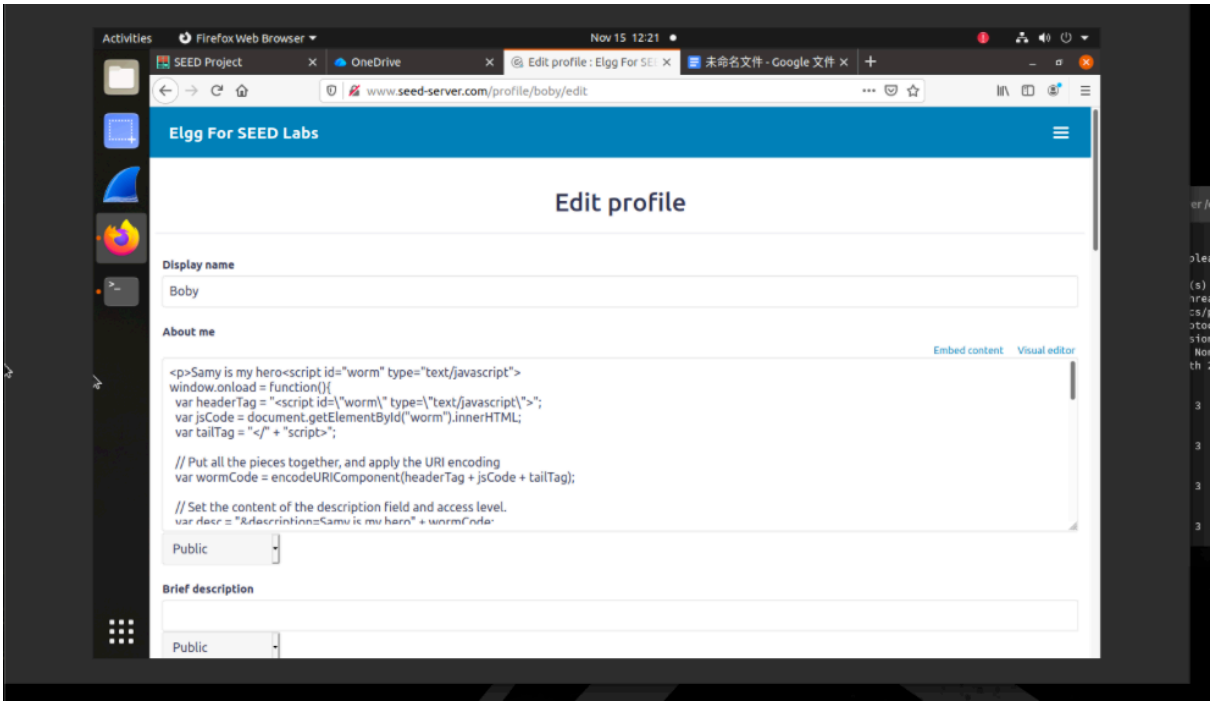
```
var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);
```

Then writes it into the victim’s profile via a POST request.

---

2) Results





3) Why the Worm Propagates Automatically

- Every victim loads the malicious JavaScript

- The exploit leverages the victim’s CSRF token to change the victim's profile.
- The script also uploaded a copy of itself to the victim profile.

As a result, each victim becomes another attacker.

This is precisely the way the original MySpace Samy Worm used to spread and affected more than 1 million users in just 20 hours.

Task 5

Content Security Policy (CSP) mitigates XSS by restricting JavaScript execution sources.

---

1) Fix XSS

SEED VM terminal

```
docker ps
```

Find container

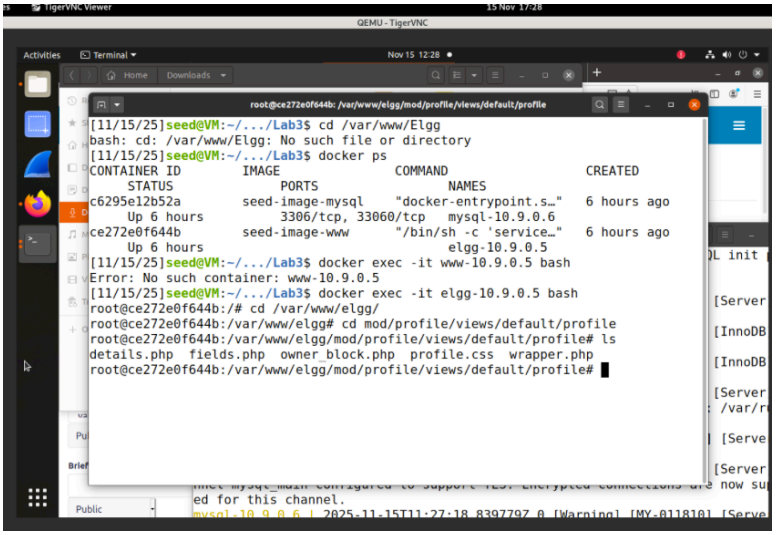
```
elgg-10.9.0.5
```

Then

```
docker exec -it elgg-10.9.0.5 bash
```

Go back container

```
cd /var/www/elgg/mod/profile/views/default/profile/
```



Find the file with

```
'value' => elgg_format_element('span', [
    'class' => $class,
], elgg_view("output/{$valtype}", [
    'value' => $value,
])),
```

Replace unsafe outputs with safe outputs

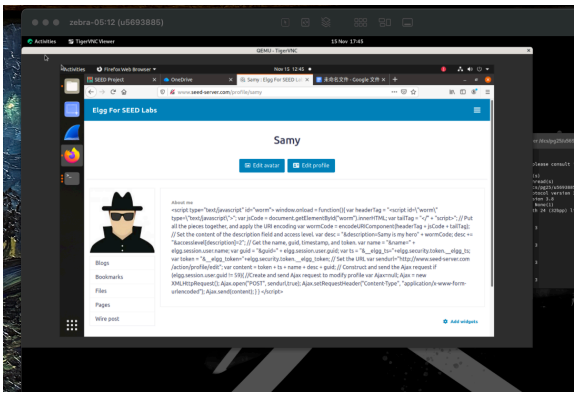
```
'value' => elgg_format_element('span', [
    'class' => $class,
], htmlspecialchars($value, ENT_QUOTES, 'UTF-8')),
```

Restart

```
docker restart elgg-10.9.0.5
```

Test

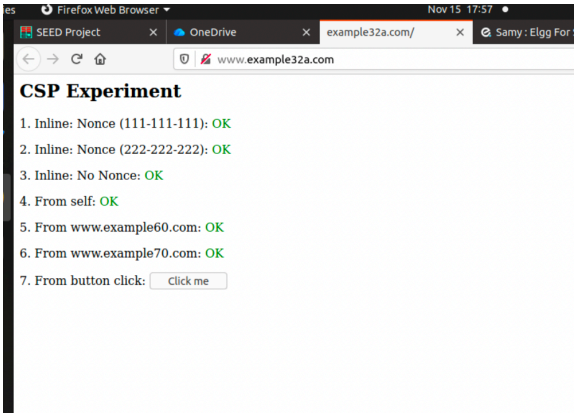
1. Save
2. go to Samy's profile

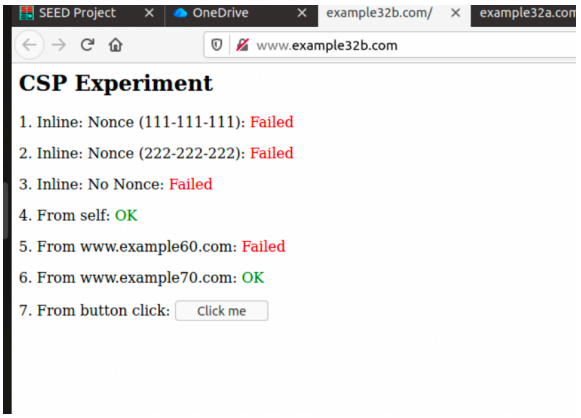


2) Observations from the Test Sites (Question4)

example32a.com

- No CSP
- All scripts run (inline + external)





example32b.com

- `script-src` only allows scripts from example60
  - Inline scripts FAIL
  - External scripts from example60 PASS

example32c.com

- Uses `nonce`
- Only scripts with correct nonce attribute execute
- All others FAIL

---

3) How to Fix example32b Areas 5 & 6 (Question5)

Modify Apache CSP header:

From:

```
script-src 'self' http://www.example60.com
```

To allow inline scripts:

```
script-src 'self' http://www.example60.com 'unsafe-inline'
```

Or safer version with a nonce:

```
script-src 'self' http://www.example60.com 'nonce-xyz123'
```

---

4) How to Fix example32c Areas 1,2,4,5,6 (Question6)

Add the correct nonce to **ALL inline** `<script>` **tags**:

Example:

```
<script nonce="xyz123">alert("OK")</script>
```

And ensure Apache header matches:

```
script-src 'self' 'nonce-xyz123'
```